

A* Algorithm

Sathyaseelan C - 241001235

Program:

```
from queue import PriorityQueue

def astar(graph, start, goal, h):
    pq = PriorityQueue()
    pq.put((0, start))

    cost = {start: 0}
    parent = {}

    while not pq.empty():
        _, node = pq.get()

        if node == goal:
            path = []
            while node in parent:
                path.append(node)
                node = parent[node]
            path.append(start)
            return path[::-1]

        for nxt, w in graph[node]:
            new_cost = cost[node] + w

            if nxt not in cost or new_cost < cost[nxt]:
                cost[nxt] = new_cost
                parent[nxt] = node
                pq.put((new_cost + h[nxt], nxt))
```

```
return None
```

```
graph = {  
    'A': [('B', 1), ('C', 4)],  
    'B': [('D', 2), ('E', 5)],  
    'C': [('F', 1)],  
    'D': [('G', 2)],  
    'E': [('G', 1)],  
    'F': [('G', 3)],  
    'G': []  
}
```

```
heuristic = {  
    'A': 7, 'B': 6, 'C': 2,  
    'D': 1, 'E': 0, 'F': 3, 'G': 0  
}
```

```
print("Shortest Path:", astar(graph, 'A', 'G', heuristic))
```

Output:

Shortest Path: ['A', 'B', 'D', 'G']